

UART Transmitter (UART TX) Design & Verification Report

Student: Huda Saad Fawzi Al-Qaey

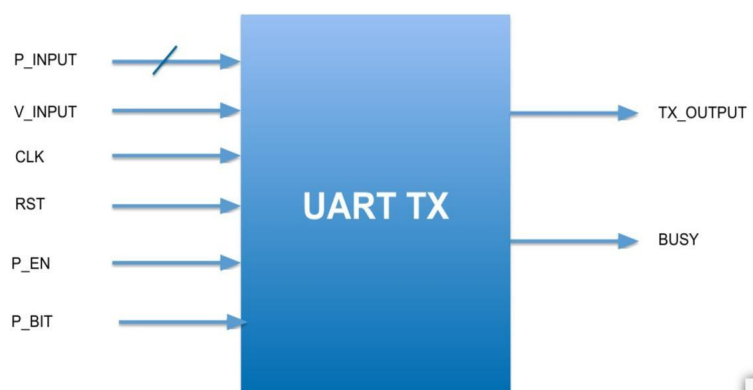
Introduction to UART

UART stands for Universal Asynchronous Receiver-Transmitter. It is a fundamental hardware communication protocol used for serial data transmission between digital systems. Unlike synchronous protocols, UART is asynchronous, meaning it does not rely on a shared clock signal between the transmitter and receiver. Instead, data is transmitted bit-by-bit sequentially over a single physical line, packaged inside standard frames that include a Start Bit(0), payload data bits, an optional Parity Bit for error detection, and a Stop Bit(1).

System Architecture & Top-Level Module (UART_TX)

The overall design is centered around the primary module UART_TX which serves as the top-level wrapper integrating four core sub-modules:

1. Finite State Machine (FSM) Control Unit: Manages state transitions, handshaking, and routing logic.
2. Serializer (PISO): Transforms parallel data into a serial bitstream.
3. Parity Calculator: Computes even or odd parity over the data payload.
4. Multiplexer (MUX): Directs the correct signal channel onto the primary output line (TX_OUTPUT).



Interface Signal Definitions:

- **P_INPUT**: Parallel data input vector to be serialized.
- **V_INPUT** : Valid input strobe triggering frame transmission.
- **CLK/ RST** : Main system clock and active-high/low reset line.
- **P_EN** : Parity Enable control bit (1): Enable parity bit, (0): Bypass parity.
- **P_BIT**: Parity Type selector (0: Even parity, 1: Odd parity).
- **TX_OUTPUT** : Physical serial output line driving transmitted data.
- **BUSY** : System status flag asserting high during active frame transmission.

Design Parameterization & Scalability :

- The top-level design and underlying sub-modules utilize Verilog parameterization (parameter DATA_WIDTH = 8).
- While configured for an 8-bit bus payload by default, the codebase is fully scalable. Adjusting DATA_WIDTH (from 8 to 16 bits) automatically updates the internal shift registers, counter dynamics, and FSM terminal counts across all modules without modifying the underlying architecture.

RTL Sub-Modules Architecture & Detailed RTL Design

Finite State Machine (FSM) Control Unit:

- **Architecture & State Encoding**: Implemented as a Moore/Mealy hybrid FSM managing five discrete operational states: IDLE (3'b000), START (3'b001), SER_DATA (3'b010), PARITY (3'b011), and STOP (3'b100).
- **State Transitions & Handshaking**: In IDLE, the FSM monitors V_INPUT. Upon detecting V_INPUT = 1, it immediately asserts BUSY = 1, transitions to START, and generates a load pulse to latch parallel data into the Serializer. After driving the Start bit, it advances to SER_DATA and asserts ser_en = 1.

- **Dynamic Path Execution:**The FSM continuously samples `ser_done` from the Serializer. Once `ser_done = 1`, it checks `P_EN`. If `P_EN = 1`, it transitions to `PARITY` for one clock cycle; if `P_EN = 0`, it bypasses `PARITY` directly to `STOP`. Finally, it drives the Stop bit, de-asserts `BUSY = 0``, and returns to `IDLE`.
- **Control Outputs:**Generates the 2-bit routing signal `mux_sel` (`2'b00`: Start, `2'b01`: Data, `2'b10`: Parity, `2'b11`: Stop/Idle) to control the top-level multiplexer output.

Parallel-In Serial-Out (PISO) Serializer Module:

- **Internal Architecture:** Built around an internal shift register sized to `DATA_WIDTH` and an internal bit-counter `log2(DATA_WIDTH)`
- **Data Latching & Shifting:**When `load = 1` is asserted by the FSM, `P_INPUT` is synchronously latched into the internal register. Upon `ser_en = 1`, data shifts out LSB-first (`P_INPUT[0]` to `P_INPUT[DATA_WIDTH-1]`), driven by `nposedge CLK`.
- **Termination Signaling:**The internal counter increments on every active clock edge during `ser_en`. Upon reaching state `DATA_WIDTH - 1` (`3'b111` for 8 bits), the module asserts `ser_done = 1` to notify the FSM that serialization is complete.

Parity Calculator Module:

- **Combinational Logic:** Computes parity dynamically over `P_INPUT` using XOR reduction logic (`^P_INPUT`).
- **Mode Selection (`P_BIT`):**Supports Even Parity (`P_BIT = 0`, parity out 1 if the number of 1s is odd) and Odd Parity (`P_BIT = 1`, parity out 1 if the number of 1s is even).
- **Gating:**Computations run combinatorially, but the output is routed to `TX_OUTPUT` strictly when selected by `mux_sel = 2'b10`.

Top-Level Multiplexer (MUX) Module:

- Routing Logic: A 4-to-1 combinational multiplexer selecting TX_OUTPUT. based on mux_sel driven by the FSM:

2'b00: Routes logic LOW (1'b0) for Start Bit.

2'b01: Routes ser_data_out from Serializer for Payload Data .

2'b10: Routes parity_out from Parity Calculator for Parity Bit .

2'b11: Routes logic HIGH (1'b1) for Stop Bit / Idle State .

Testbench Implementation, Verification Methodology and Debugging Journey

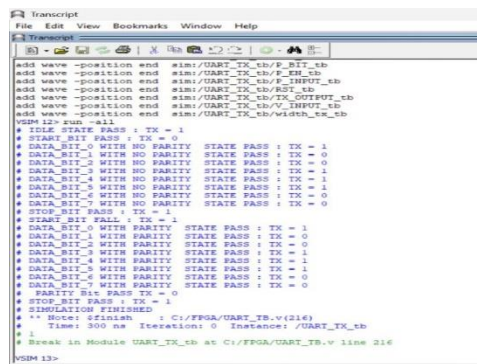
To ensure complete verification coverage, the system was evaluated using top-level testbench simulations, followed by sub-module isolation tests, and finally targeted debugging iterations on delay controls.

Phase 1: Initial Top-Level Testing and Waveform Analysis

Top-level verification began using the master testbench (UART_TB.v) to sample output signals on TX_OUTPUT.

Transcript Baseline: In the initial baseline run, all payload data bits (DATA_BIT_0 through DATA_BIT_7), and the Stop Bit achieved PASS status in the script transcript. However, the Start Bit reported a FAIL in the transcript script (specifically during the second frame transmission with parity enabled).

Visual inspection of ModelSim waveforms was conducted to compare physical signal transitions against the transcript output.

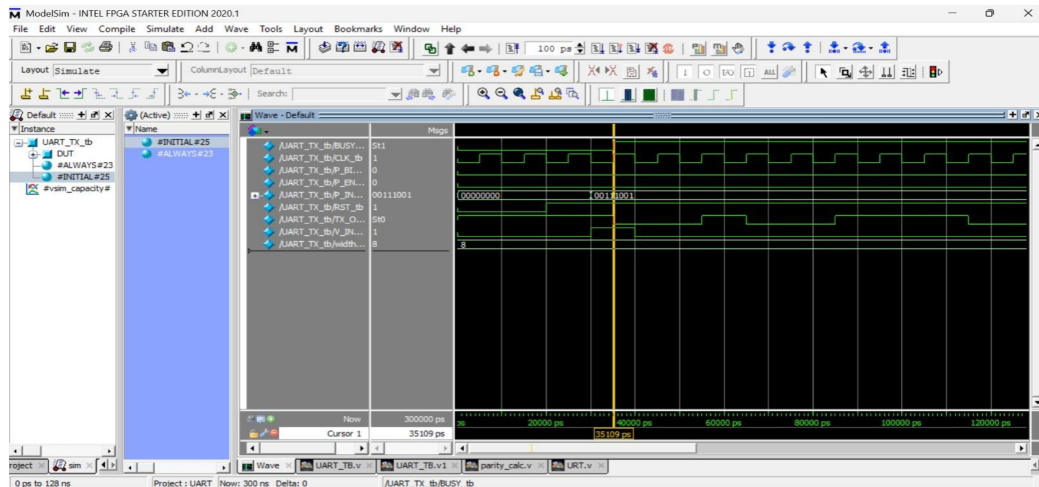


```
Transcript
File Edit View Bookmarks Window Help

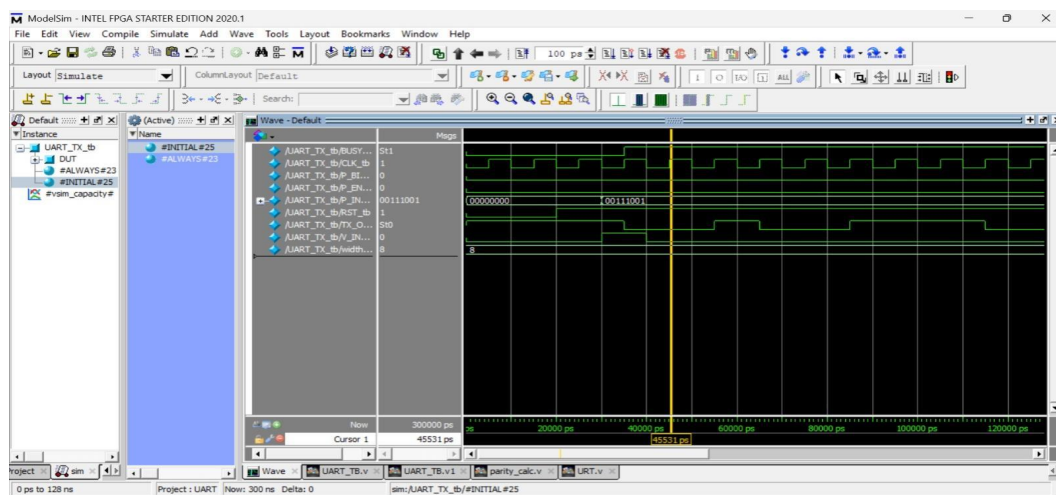
add wave -position end a1m:/UART_TB.cb/P_RIT_cb
add wave -position end a1m:/UART_TB.cb/P_FIR_cb
add wave -position end a1m:/UART_TB.cb/P_INPUT_cb
add wave -position end a1m:/UART_TB.cb/RSI_cb
add wave -position end a1m:/UART_TB.cb/TX_OUTPUT_cb
add wave -position end a1m:/UART_TB.cb/V_INPUT_cb
add wave -position end a1m:/UART_TB.cb/STATUS_RK_cb
VSIHM 12> run -all
# IDLE STATE PASS : TX = 1
# START_BIT PASS : TX = 0
# DATA_BIT_0 WITH NO PARITY STATE PASS : TX = 1
# DATA_BIT_1 WITH NO PARITY STATE PASS : TX = 0
# DATA_BIT_2 WITH NO PARITY STATE PASS : TX = 0
# DATA_BIT_3 WITH NO PARITY STATE PASS : TX = 1
# DATA_BIT_4 WITH NO PARITY STATE PASS : TX = 1
# DATA_BIT_5 WITH NO PARITY STATE PASS : TX = 1
# DATA_BIT_6 WITH NO PARITY STATE PASS : TX = 0
# DATA_BIT_7 WITH NO PARITY STATE PASS : TX = 0
# STOP_BIT PASS : TX = 1
# START_BIT FAIL : TX = 1
# DATA_BIT_0 WITH PARITY STATE PASS : TX = 1
# DATA_BIT_1 WITH PARITY STATE PASS : TX = 0
# DATA_BIT_2 WITH PARITY STATE PASS : TX = 0
# DATA_BIT_3 WITH PARITY STATE PASS : TX = 1
# DATA_BIT_4 WITH PARITY STATE PASS : TX = 1
# DATA_BIT_5 WITH PARITY STATE PASS : TX = 1
# DATA_BIT_6 WITH PARITY STATE PASS : TX = 0
# DATA_BIT_7 WITH PARITY STATE PASS : TX = 0
# PARITY_RLS PASS : TX = 0
# STOP_BIT PASS : TX = 1
# SIMULATION FINISHED
# ** Note: Finish : C:/FPGA/UART_TB.v(216)
# Time: 500 ns Iteration: 0 Instance: UART_TB.cb
# Break in Module UART_TB.cb at C:/FPGA/UART_TB.v line 216
VSIHM 13>
```

Detailed Waveform Inspection Breakdown:

Start Bit Case (First posedge CLK): When V_INPUT = 1 and BUSY = 1, the system enters the Start bit condition. The waveform shows TX_OUTPUT = 0, matching expected hardware logic.



Data Bit Case (Data Bit 0): When V_INPUT = 0 and BUSY = 1, the system transitions into serial transmission, and TX_OUTPUT correctly displays the payload data starting at DATA_BIT_0.



Second Frame Transmission (With Parity Enabled): After the first frame finishes, BUSY drops to 0 following the Stop bit. Then V_INPUT goes High (1)

ModelSim - INTEL FPGA STARTER EDITION 2020.1

File Edit View Compile Simulate Add Wave Tools Layout Bookmarks Window Help

Layout Simulate ColumnLayout Default

Search:

Wave - Default

Instance

- UART_TB
 - OUT
 - PALWAYS#23
 - #INITIAL#25
 - #vram_capacity#

Wave - Default

Signal	Value	Time
#UART_TX_B#BUSY...	011	00111001
#UART_TX_B#CLK_B#	1	
#UART_TX_B#P_B#...	0	
#UART_TX_B#P_EN...	1	
#UART_TX_B#P_IN...	1	
#UART_TX_B#RST_B#	1	
#UART_TX_B#TX_O...	010	
#UART_TX_B#V_IN...	1	
#UART_TX_B#width...	8	

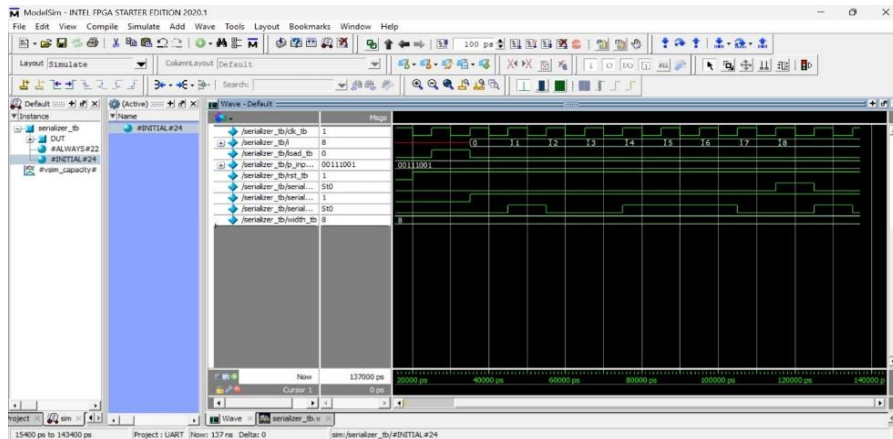
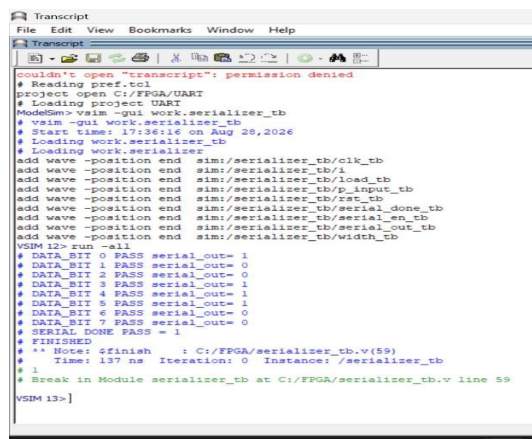
Now: 300000 ps

Cursor 1: 165075 ps

Project: UART Now: 300 ns Delta: 0

To confirm whether the Start bit transcript failure originated from sub-module RTL logic or top-level control integration, testing was paused at the top level to run standalone unit testbenches for both core sub-modules.

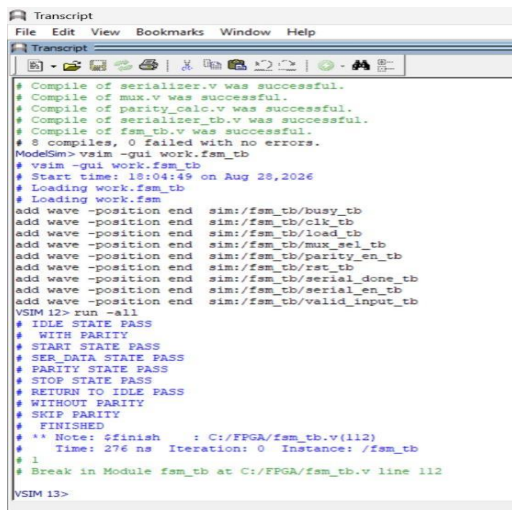
Implementation: Built a dedicated testbench applying an 8-bit parallel vector P_INPUT, triggering load, and monitoring ser_data_out and ser_done. Result: All unit test cases passed. Data shifted out LSB-first in synchronization with posedge CLK, and ser_done asserted cleanly without errors.



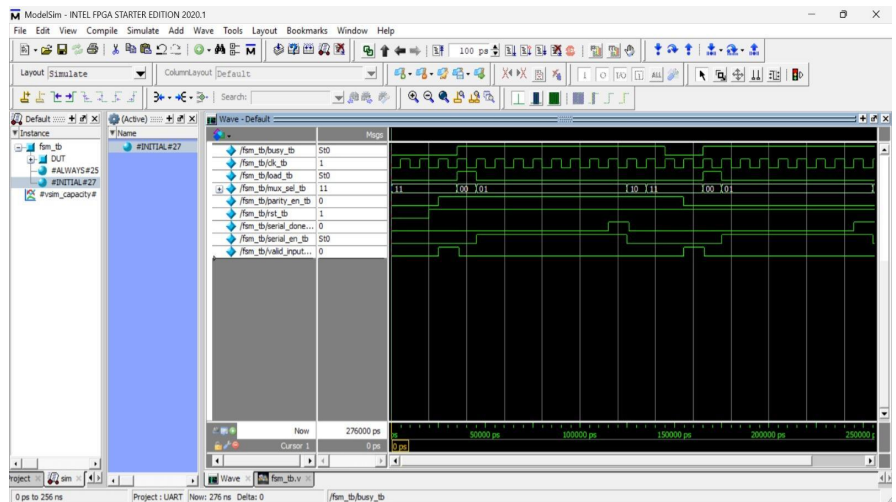
Standalone FSM Unit Verification (fsm_tb)

Implementation: Constructed a standalone testbench driving V_INPUT, P_EN, and ser_done across all states (IDLE, START, SER_DATA, PARITY, STOP).

Result: All unit test cases passed. State transitions, BUSY flag control, and mux_sel routing signals operated as specified.



```
ModelSim> vsim -gui work.fsm_tb
# Compile of fsm_tb.v was successful.
# Compile of fsm_tb.v was successful.
# 8 compiles, 0 failed with no errors.
ModelSim> vsim -gui work.fsm_tb
# Start time: 18:04:49 on Aug 28, 2026
# Loading work.fsm_tb
# Loading work.fsm
add wave -position end sim:/fsm_tb/busy_tb
add wave -position end sim:/fsm_tb/clk_tb
add wave -position end sim:/fsm_tb/load_tb
add wave -position end sim:/fsm_tb/mux_sel_tb
add wave -position end sim:/fsm_tb/parity_en_tb
add wave -position end sim:/fsm_tb/rst_tb
add wave -position end sim:/fsm_tb/serial_done_tb
add wave -position end sim:/fsm_tb/serial_en_tb
add wave -position end sim:/fsm_tb/valid_input_tb
VSIM 12> run -all
# IDLE STATE PASS
# WITH PARITY
# START STATE PASS
# SER_DATA STATE PASS
# PARITY STATE PASS
# STOP STATE PASS
# RETURN TO IDLE PASS
# WITHOUT PARITY
# SKIP PARITY
# FINISHED
# Note: Finish : C:/FPGA/fsm_tb.v(112)
# Time: 276 ns Iteration: 0 Instance: /fsm_tb
# Break in Module fsm_tb at C:/FPGA/fsm_tb.v line 112
VSIM 13>
```



Phase 3: Top-Level Debugging and Delay Control Iterations

With standalone unit tests passing for both the FSM and Serializer, multiple debugging attempts were executed on the top-level testbench (UART_TB.v) to resolve the Start bit transcript mismatch.

Attempt 1: Modifying Delay Control (#10 to #15)

Action: Changed sampling assertion delay from #10 to #15 in the testbench.

Result: The Start bit remained FAIL. Payload data bits produced alternating pass and fail outcomes (some data bits passed while others failed).

Key Finding: Parity Bit and Stop Bit maintained PASS status.

Attempt 2: Clock Synchronization Before Start Bit Only

Action: Added posedge CLK synchronizer strictly before the Start bit assertion check.

Result:Start Bit: Changed to PASS.

Data Bits: Produced mixed results (DATA_BIT_0 PASS, DATA_BIT_1 FAIL, DATA_BIT_2 PASS, DATA_BIT_3 FAIL, DATA_BIT_4 PASS, DATA_BIT_5 PASS, DATA_BIT_6 FAIL, and DATA_BIT_7 PASS).

Parity and Stop Bits: Maintained PASS status.

[illegible]

Attempt 3: Clock Synchronization Across Data Bits (posedge CLK)

Action: Inserted posedge CLK synchronizers before checking every data bit condition (instead of checking only before the Start bit).

Result:Start Bit: Changed to PASS.

Data Bits: Produced mixed results (DATA_BIT_0 PASS, DATA_BIT_1 FAIL, DATA_BIT_2 PASS, DATA_BIT_3 FAIL, DATA_BIT_4 PASS, DATA_BIT_5 PASS, DATA_BIT_6 FAIL, and DATA_BIT_7 PASS).

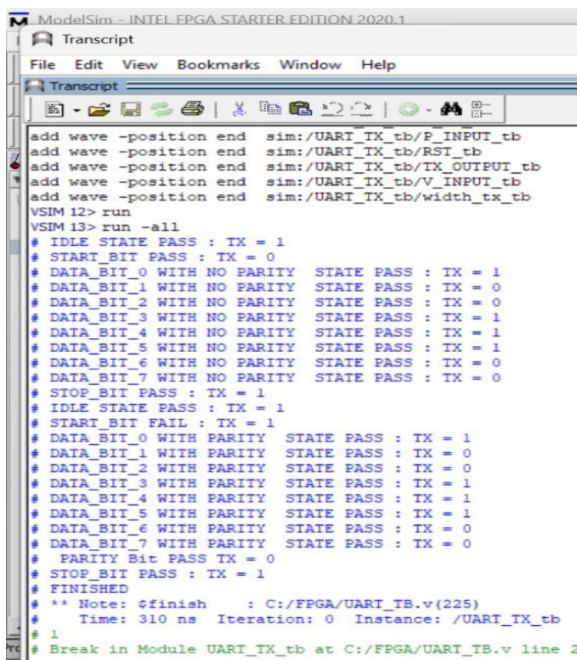
Parity and Stop Bits: Maintained PASS status.

Key Observation: Applying posedge CLK synchronizers across all individual data bit conditions yielded the exact same outcome as Attempt 2, with no difference observed in the transcript output.

Attempt 4: BUSY and TX_OUT Signal Condition Checking (TX_OUTPUT == 1 && BUSY == 0)

Action: Added assertion logic to check the specific condition where TX_OUTPUT = 1 and BUSY == 0 to verify TX_OUTPUT driving during idle state.

Result: On the waveform, signals behaved correctly. However, in the transcript, the Start bit still logged FAIL for the second frame (with parity enabled), while all payload data bits (DATA_BIT_0 through DATA_BIT_7), the Parity Bit, and the Stop Bit, Idle state, passed cleanly.



```
ModelSim - INTEL FPGA STARTER EDITION 2020.1
Transcript
File Edit View Bookmarks Window Help
Transcript
add wave -position end sim:/UART_TX_tb/P_INPUT_tb
add wave -position end sim:/UART_TX_tb/RST_tb
add wave -position end sim:/UART_TX_tb/TX_OUTPUT_tb
add wave -position end sim:/UART_TX_tb/V_INPUT_tb
add wave -position end sim:/UART_TX_tb/width_tx_tb
VSIM 12> run
VSIM 13> run -all
# IDLE STATE PASS : TX = 1
# START_BIT PASS : TX = 0
# DATA_BIT_0 WITH NO PARITY STATE PASS : TX = 1
# DATA_BIT_1 WITH NO PARITY STATE PASS : TX = 0
# DATA_BIT_2 WITH NO PARITY STATE PASS : TX = 0
# DATA_BIT_3 WITH NO PARITY STATE PASS : TX = 1
# DATA_BIT_4 WITH NO PARITY STATE PASS : TX = 1
# DATA_BIT_5 WITH NO PARITY STATE PASS : TX = 1
# DATA_BIT_6 WITH NO PARITY STATE PASS : TX = 0
# DATA_BIT_7 WITH NO PARITY STATE PASS : TX = 0
# STOP_BIT PASS : TX = 1
# IDLE STATE PASS : TX = 1
# START_BIT FAIL : TX = 1
# DATA_BIT_0 WITH PARITY STATE PASS : TX = 1
# DATA_BIT_1 WITH PARITY STATE PASS : TX = 0
# DATA_BIT_2 WITH PARITY STATE PASS : TX = 0
# DATA_BIT_3 WITH PARITY STATE PASS : TX = 1
# DATA_BIT_4 WITH PARITY STATE PASS : TX = 1
# DATA_BIT_5 WITH PARITY STATE PASS : TX = 1
# DATA_BIT_6 WITH PARITY STATE PASS : TX = 0
# DATA_BIT_7 WITH PARITY STATE PASS : TX = 0
# PARITY Bit PASS TX = 0
# STOP_BIT PASS : TX = 1
# FINISHED
** Note: $finish : C:/FPGA/UART_TB.v(225)
Time: 310 ns Iteration: 0 Instance: /UART_TX_tb
1
Break in Module UART_TX_tb at C:/FPGA/UART_TB.v line 2
```

Final Decision

Because altering testbench sampling delays and clock edge alignment checks introduced artificial failures across data payload bits, the top-level testbench code was retained in its baseline state. In this baseline configuration, all payload data bits (0 through 7), parity calculations, and stop bits pass in the transcript, while the single Start bit failure for the second frame in the transcript remains an identified testbench strobe control issue under further refinement.

